

# Club App: Week 1 Tech Plan

SIBC Amazon Project | Dev team: York, Gabriel, Eduardo, Brendan

*Draft for discussion, October 4, 2026. Status: everything in this document is proposed. Nothing is built yet.*

---

Short version: the current plan is a solid start, but a few pieces will hurt us in front of Amazon, and a couple more will eat the 6 weeks. This document covers the pushback first, then answers each of the meeting questions, then the architecture diagram and the roadmap.

## 1. Pushback on the current plan

**Drop the Gemini API.** We are presenting to Amazon. Using Google's model when Amazon Bedrock exists looks like we didn't consider the native option. If we do any AI at all, use Bedrock (Amazon Nova Micro is cheap and fast enough to tag events). AI should be a stretch goal anyway; see the next point.

**The For You feed should be rule-based, not AI.** This is also our strongest defense line. A brand-new app has no interaction data, so a machine-learning recommender has nothing to learn from (the cold-start problem). A transparent scoring rule (interest match + clubs you're in + how soon the event is + RSVP count, with past events excluded) works on day one. On the "future" slide we say: once we have real interaction data, we would move to Amazon Personalize, which is built on the same recommendation tech as Amazon.com.

**Calendars don't go in S3.** S3 stores files (club docs, event flyers). Events are structured data, so they go in DynamoDB. The .ics file is generated on demand by a Lambda function from DynamoDB data. It is never stored.

**Cut two-way Google Calendar sync.** OAuth with Google, sync conflicts, and token refresh are a project by themselves. Do .ics export instead. Stretch goal: a subscribable .ics feed URL per user, which Google, Apple, and Outlook all support natively. That gets 90% of the value for 10% of the work.

**Cut videos and the Instagram-style media feed.** Video means transcoding and storage costs, and it doesn't serve the prompt. Event cards with one flyer image are enough.

**Don't ignore finances.** The prompt explicitly says "events, finances, and communication." The current plan only lists "Financials" under documents, and the judges will notice if finances are missing. Build a minimal ledger: officers log income and expenses with a

category, and the club page shows the running balance. It's cheap to build and directly answers the prompt.

**Rethink hamburger + bottom tabs.** Right now Discover and Calendar, our two most differentiating features, are buried in a hamburger menu, which is where features go to die. Proposal: 5 bottom tabs, like Instagram: Home (your clubs), Discover, Calendar, Inbox, Profile. The hamburger can hold settings and club documents. This is a UX call for the team to debate.

**Keep “no independent messaging.”** Good call. The inbox is one-way: officers post announcements, members receive them. DMs and group chat are a trap (real-time sync, moderation, read receipts). Make sure the whole team agrees that the “unified inbox” means announcements, not chat, so nobody starts building chat in week 4.

## 2. Answers to the meeting questions

### Languages

TypeScript everywhere: frontend, backend definitions, and Lambda functions. One language means anyone can review anyone's PR, and data types are shared between frontend and backend automatically.

### Frontend

React with Vite, built as a mobile-first responsive web app (a PWA, so it can be added to a phone's home screen). Avoid React Native here: the demo will happen on a laptop and projector, Amplify Hosting serves web apps directly, and app-store builds are a time sink. A PWA with bottom tabs looks and feels like an app on a phone anyway.

### Backend

AWS Amplify Gen 2. We define the data model, auth, storage, and functions in TypeScript, and Amplify deploys them as Cognito, AppSync, DynamoDB, S3, and Lambda through AWS CDK. Defense line: “This is AWS's recommended path for full-stack apps. Our infrastructure is code, version-controlled in GitHub, and every developer gets an isolated cloud sandbox.” The per-developer sandbox (npx ampx sandbox) is a big deal for a 4-person team, because nobody breaks anyone else's backend.

### Hosting

Amplify Hosting, connected to GitHub. Pushing to main auto-deploys. Use a dev branch for testing and main for the demo URL.

### What data is stored, and how

| Data                                                      | Where    | Notes                                               |
|-----------------------------------------------------------|----------|-----------------------------------------------------|
| User (name, year, interest tags)                          | DynamoDB | Linked to the Cognito user                          |
| Club (name, description, category, founded year)          | DynamoDB | Member count is derived from memberships            |
| Membership (user, club, role: member/officer)             | DynamoDB | Indexed by user and by club                         |
| Event (club, title, start/end, location, tags, flyer key) | DynamoDB | Index on club + start time                          |
| RSVP (user, event)                                        | DynamoDB | Feeds the popularity score                          |
| Announcement (club, body, created at)                     | DynamoDB | The inbox is a query over your clubs' announcements |
| Ledger entry (club, amount, type, category, date, note)   | DynamoDB | Officers only                                       |
| Club documents and event flyers                           | S3       | Metadata row in DynamoDB, file in S3                |

**Permissions (solve this week).** “Only officers of this club can post events for it” doesn't fit cleanly into Cognito groups, because roles are per club, not global. Recommendation: anyone signed in can read; anything officer-only (create event, post announcement, add ledger entry) goes through a custom mutation backed by a Lambda function that checks the Membership role first. Simple, secure, and easy to explain.

**Table design tradeoff.** Amplify Data creates one DynamoDB table per model, not the “single-table design” some AWS engineers prefer. If asked: “We traded some query efficiency for development speed. At our scale it doesn't matter, and the access patterns are documented if we need to migrate.”

### AWS services: MVP vs. future

| In the MVP      | Why                                                                      |
|-----------------|--------------------------------------------------------------------------|
| Amplify Hosting | Hosting + CI/CD from GitHub                                              |
| Cognito         | Sign-in. A pre-signup Lambda trigger restricts sign-up to @nd.edu emails |

| In the MVP | Why                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| AppSync    | The GraphQL API Amplify generates, with auth rules built in                                                                                   |
| DynamoDB   | All structured data                                                                                                                           |
| Lambda     | Feed ranking, .ics export, officer permission checks, email triggers                                                                          |
| S3         | Club documents and flyers, uploaded through presigned URLs                                                                                    |
| SES        | Announcement emails. Stays in sandbox mode, so it only sends to verified addresses; verify the team's emails for the demo and say so if asked |
| CloudWatch | Logs come free with Lambda                                                                                                                    |

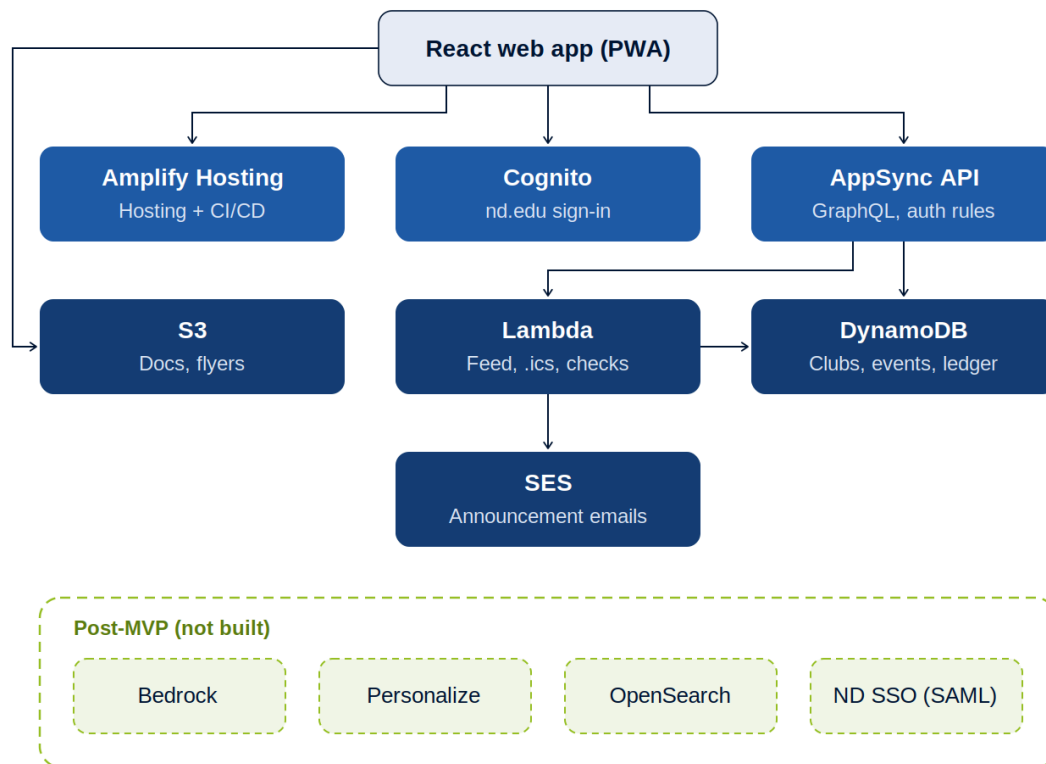
| With more time or budget | Pitch                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------------|
| Amazon Personalize       | ML recommendations once real interaction data exists                                            |
| Amazon Bedrock           | Auto-tag events from descriptions, summarize long announcements                                 |
| Cognito SAML federation  | Notre Dame single sign-on instead of email/password                                             |
| OpenSearch Service       | Full-text search at multi-campus scale. Overkill for ~500 clubs, where in-app filtering is fine |
| EventBridge Scheduler    | Event reminder emails 24 hours before                                                           |
| Rekognition / Comprehend | Moderation of flyers and announcements                                                          |
| QuickSight               | Engagement analytics dashboards for student activities offices                                  |

Set up an AWS Budgets alarm (around \$10) on day one. Almost everything here fits in the free tier, but a forgotten resource can still cost money.

### Features: MVP vs. cut

- **MVP:** @nd.edu sign-in, club pages with stats, join/leave, the officer role, events with RSVP, announcements plus the inbox, the Discover feed, search and filter, a calendar view with .ics export, the finance ledger, document upload, and a profile with interests.
- **Stretch:** announcement emails through SES, a subscribable calendar URL, event reminders, and Bedrock tagging.
- **Cut:** Google Calendar sync, video, DMs, and dues/payment collection (real money means a payment provider and compliance work, which is out of scope).

### 3. Architecture



A typical request: the app signs the user in through Cognito, then calls AppSync. AppSync reads and writes DynamoDB directly for simple operations and calls a Lambda function for logic, like feed ranking, .ics generation, or “is this user an officer of this club?” File uploads go straight from the browser to S3 using a short-lived presigned URL, so files never pass through the API. Lambda sends announcement emails through SES. Everything in the dashed green box is a future-work talking point, not part of the build.

### 4. Six-week roadmap

#### ~~Roles (swap based on strengths)~~

- ~~**York (lead):** owns the architecture and data model, reviews every PR, handles integration and the demo script.~~
- ~~**Gabriel (backend/data):** Amplify Data schema, indexes, Lambda functions (feed ranker, .ics export, officer checks), the seed script.~~

- **Eduardo (frontend):** app shell and tab navigation, club pages, Discover, the Calendar UI, visual polish.
- **Brendan (auth + features):** Cognito and the @nd.edu trigger, announcements/inbox, the ledger, S3 documents, SES.

| Week | Dates        | Goal                    | Done when                                                                                                                               |
|------|--------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 00   | Oct 5–11     | Design freeze           | Diagram, data model, frozen MVP feature list, wireframes. Repo plus Amplify Gen 2 scaffold. All four people can run their own sandbox   |
| 01   | Oct 12–18    | Foundation              | Sign-in works with @nd.edu only, schema deployed, seed script loads ~30 realistic ND clubs, tab navigation, club pages render real data |
| 02   | Oct 19–25    | Core loop               | Join/leave, officer role enforced, create events, RSVP, announcements, the inbox, search/filter                                         |
| 03   | Oct 26–Nov 1 | Differentiators         | Discover feed ranking, calendar view, .ics export, ledger, document upload                                                              |
| 04   | Nov 2–8      | Feature freeze + polish | Stretch goals only if weeks 1–4 are done. SES emails, bug bash, main deployed to the demo URL, realistic seed data                      |
| 05   | Nov 9–15     | Presentation            | Rehearsed demo path, backup screen recording of the demo, architecture and cost slides, Q&A prep                                        |

## Rules that will save us

1. Feature freeze at the end of week 5 is non-negotiable. Week 6 is for rehearsal, not coding.
2. Check when fall break lands (usually mid-to-late October, likely weeks 2–3). If people are away, move work earlier, not later.
3. Seed data matters. A demo with three clubs called “Test Club” looks dead; 30 real-sounding ND clubs with upcoming events looks like a product.
4. Record a backup video of the demo by week 6. Venue Wi-Fi will fail exactly once.

## 5. Deliverables for this week

The architecture diagram (redrawn in slide colors), the data model table, the MVP-vs-future services table, the frozen feature list, and the scaffolded repo. If York signs off on the stack

by mid-week, Gabriel can start the schema immediately, since everything else depends on it.